Number System

How the conversions work

Conversion	Method
Decimal → Binary	Repeatedly divide by 2, record remainders, read bottom-up
Binary → Decimal	Multiply each bit by its power of 2, sum them
Decimal → Hex	Repeatedly divide by 16, record remainders (10–15 → A–F)
Hex → Decimal	Multiply each digit by its power of 16, sum them
Binary → Hex (shortcut)	Group bits into 4s (nibbles) from the right, convert each nibble directly
Hex → Binary (shortcut)	Expand each hex digit into exactly 4 binary digits

Reference Table

Decimal	Binary	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Nibble Shortcut

The **nibble shortcut** is the fast way to convert between binary and hex without going through decimal at all.

Why it works: Hex has 16 symbols (0–9, A–F), and 16 = 2^4. So every single hex digit corresponds to *exactly* 4 binary digits — no more, no less. A group of 4 bits is called a **nibble** (half a byte).

Binary → Hex: Split the binary number into groups of 4 bits starting from the right, then convert each group to its hex digit using the reference table.

Example — Binary `10110101` → Hex:

```
1011  0101
 ↓     ↓
 B     5
```

Answer: `B5`

Hex → Binary: Reverse it — expand each hex digit into its 4-bit binary equivalent and concatenate.

Example — Hex `A3` → Binary:

```
A    3
↓    ↓
1010 0011
```

Answer: `10100011`

File Types

Text	Binary
.py, .csv, .json, .html, .txt	.jpg, .mp3, .mp4, .exe, .zip, .pkl, .docx

Text files

- Source code (any Programming Language)
- CSV : Comma Separated Values
- **Any data that humans need to read**

ASCII

If a text file contains only basic English letters, digits, punctuation, and common control characters like newline, then all those characters are in the ASCII range.

ASCII includes:

- `A–Z`, `a–z`
- `0–9`
- Punctuation like `. , ! ? ; : ( )`
- Space
- Control characters like newline (`\n`) and tab (`\t`)

Later we'll learn about Unicode and UTF-8, which allow text files to contain characters like `é`, `你`, and 😀

Why media specifically needs binary

- A pixel's colour value or a sound sample is just a number — there's no "letter" for it.
- The sharper reason: text mode can silently translate certain byte patterns (treating them as line endings). If raw media data happens to contain such a pattern, opening/saving it in text mode can corrupt the file
- Bonus: storing raw bytes is also far more compact than spelling numbers out as text characters.

Other places binary wins over text

compiled/executable code, serialized objects for speed, compressed or encrypted data, large numeric datasets where size/speed matter.